

- Yuandong Tian, Yiping Wang, Beidi Chen, and Simon Du. 2023. Scan and snap: Understanding training dynamics and token composition in 1-layer transformer. *arXiv preprint arXiv:2305.16380*.
- Curt Tigges, Oskar John Hollinsworth, Atticus Geiger, and Neel Nanda. 2023. Linear representations of sentiment in large language models. *arXiv preprint arXiv:2310.15154*.
- Eric Todd, Millicent L Li, Arnab Sen Sharma, Aaron Mueller, Byron C Wallace, and David Bau. 2023. Function vectors in large language models. *arXiv preprint arXiv:2310.15213*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Lean Wang, Lei Li, Damai Dai, Deli Chen, Hao Zhou, Fandong Meng, Jie Zhou, and Xu Sun. 2023. [Label words are anchors: An information flow perspective for understanding in-context learning](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9840–9855, Singapore. Association for Computational Linguistics.
- Quan-shi Zhang and Song-Chun Zhu. 2018. Visual interpretability for deep learning: a survey. *Frontiers of Information Technology & Electronic Engineering*, 19(1):27–39.

A The Rank of The Last Layer

In [Section 4.1](#), we delve into the observation that each gradient matrix has a rank equal to the length of the edited prompt (annotated by n), except for the last layer’s ones. In this section, we explain why the last layer’s MLP matrices are always rank-1.

The backward pass, applied to the final loss score (L , [Section 3.2](#)), generates a computational graph that is reversed in direction compared to the forward pass [Section 3.2](#). It begins with the loss score and the matrices of the last layer, proceeding in reverse order until reaching the matrices of the first layer. This computational graph encapsulates every hidden state and intermediate result that contributed to the final prediction, which is the output of the last layer for the last token in the prompt.

One might initially assume that, since the last prediction was formed by the input of the last token, only its hidden states would be involved in this computational graph. However, due to the attention mechanism, hidden states from previous forward passes can be recalled and utilized in subsequent forward passes, contributing to all the tokens that follow them in the prompt. The last hidden state to be recalled using the attention modules is called at the model’s last layer’s attention module, which precedes an MLP module in sequential architectures, such as GPT2 and Llama-7B. Hence, in every layer, MLP inputs in the reverse computational graph comprise all individual intermediate inputs x_i from the forward pass of each token in the prompt. However, at the last layer, the only input included is the one belonging to the last prompt token x_n . For this reason, also only the VJP of the last token, δ_n , is included in the reconstruction of the gradients of the last MLP layer, while the δ_i for all the other tokens from the prompt are not included (or more correctly, they are equal to the zero vectors).

When constructing the gradients using x_i and δ_i , the rank of each layer is equal up to the number of x_i, δ_i involved in the computational graph (assuming linear independence [Section 4.1](#)). This implies that all layer matrices are formed by $\frac{\partial L}{\partial W} = \sum_{i=1}^n x_i^\top \cdot \delta_i$ except for the last layer, which is constructed with $\frac{\partial L}{\partial W} = x_n^\top \cdot \delta_n$, which is rank-1.

In our study, especially in our figures and tables, we decided to include all the vectors of the last layer, including those from tokens which are not the prompt’s last, which are thus equal to zero vectors. This approach is also the reason for the observed

changes in the behavior of the gradients in some figures. For example, in [Figure 6](#) we can see that all the graphs (except for the last token’s) converge to the same value at the last layer. The reason for this is that they are all equal to the zero vector. In [Figure 11](#) we see the LL projection of the VJPs from the model’s last layer, which are equal to projecting the zero vector.

B Layer Norm’s Backward Step

Layer Norms ([Ba et al., 2016](#)) are used in transformers to scale their activations, thereby speeding up the training process and making it more stable. Although they contain learnable weights, they are usually omitted in interpretability literature. For completeness, we demonstrate how to calculate the derivative of the Layer Norm component and examine its effect on the backward pass.

Given an input vector $x \in \mathbb{R}^d$, Layer Norm is defined as the following scaled dot product:

$$\text{LayerNorm}(x) = \alpha \odot \frac{x - \mu}{\sigma} + \beta \quad (13)$$

where μ and σ are the mean and standard deviation of the input x respectively. $\alpha, \beta \in \mathbb{R}^d$ are scaling and shifting (bias) parameters, which are learnable.

In a general fine-tuning process, as well as in our experiment in [Section 6](#), Layer Norms are frozen (their parameters are not updated). However, during the backward pass, we do apply the backward step through the Layer Norm, necessitating the calculation of the derivative of the Layer Norm with respect to the forward pass inputs.

Given the VJP $\delta \in \mathbb{R}^d$ up to a given Layer Norm, the backward step of the Layer Norm that produces $\hat{\delta}$ is defined by:

$$\gamma_1 = (x - \mu) \odot \sigma \in \mathbb{R}^d \quad (14)$$

$$\gamma_2 = \delta \odot \alpha \in \mathbb{R}^d \quad (15)$$

$$\hat{\delta} = (\gamma_2 - \mu_{(\gamma_1)} - \gamma_1 \mu_{(\gamma_2 \odot \gamma_1)}) \odot \sigma \in \mathbb{R}^d \quad (16)$$

where $\mu_{(v)}$ is the mean of the vector v .

We view this backward step as a mirroring process to the normalization step performed during the forward pass. Furthermore, our results in [Section 6](#), conducted on full transformer models, demonstrate that the effect of Layer Norms on the behavior (information embedded) that was predicted by the theoretical analysis [Section 5](#) is negligible.

C Why Decomposed Gradient Analysis Makes Sense

In [Section 3.3](#), [Section 4.2](#) we establish our interpretation of gradients via spanning sets. This approach is based on the understanding that each neuron in the gradient matrix is formed by the linear combination of x_i (the forward pass’s intermediate inputs) or δ_i (VJPs, the backward pass’s hidden states). In this section, we aim to illustrate, through a singular example, why analyzing a gradient matrix through its spanning set is more informative and simpler compared to attempting to analyze the full gradient matrix.

We use GPT2-medium (24 layers and 330M parameters) for our examination. We examine the MLP gradients using the prompt “Lionel Messi plays for”, to which the model responds with “Barcelona”. We edit the model with a single backward pass to respond “Paris”. In the case of this model, each MLP matrix comprises 4096 neurons. Consequently, to apply the Logit Lens (LL) projection to a particular gradient matrix, the process needs to be applied 4096 times.

We start by analyzing FF_2 from layer 14 . In [Table 2](#) we present samples of gradient neurons’ projections by LL. In [Section 4.2](#), we elaborate on how each neuron is formed by multiplying an interpretable vector by a coefficient (δ_i and x_i), which in turn dictates its norm. We group these neurons based on their norms, unveiling shared projections among neurons within the same norm group. To underscore the proximity of certain neurons to the zero vector, we also include the projection of the zero vector. Expanding [Table 2](#) to every individual neuron in the matrix is impractical, given the challenge of reading a table with 4096 rows. Instead, we use plots [Figure 7](#): the initial plot presents the LL intersection, measuring the extent of overlap between the top 100 most probable tokens from two vectors, while the second shows the cosine similarity between the vectors.

We repeat the process after sorting the gradient neurons according to their norms [Figure 8](#). The gradients with the higher norms, are almost identical to the last VJP (δ_n), with alignment extending up to the sign of the vectors. In [Table 3](#) LL reveals that these VJPs project “Paris”. Gradients with low norms may appear correlated with parts of the spanning set’s vectors (FF_2 ’s δ_i), yet they are more correlated to the zero vector, emphasizing that these neurons do not update the model’s

weights (induce minimal change).

In addressing color shifting, we see around index 500 from the right that this is where the activations change sign from positive to negative. The negative learning rate causes the positive activation to add “Paris” into those neurons, while the negative activation reduces “Paris”. In both cases, the process causes the model to add “paris” in the same direction ([Section 5.2](#)).

We repeat this type of analysis with FF_1 . We remind that our interpretation for this layer’s spanning set is its inputs x_i [Section 4.2](#). Again, we see the alignment between individually analyzing the neurons of the gradients and the spanning set [Table 4](#), [Table 5](#), [Figure 9](#).

In conclusion, In this section, we demonstrate the converging results of analyzing individual gradient neurons via LL and the spanning sets interpretation. The aim of this analysis is to emphasize the efficacy of employing these spanning sets to simplify experiments involving a vast number of vectors (neurons) into a much smaller, representative subset. The advantage of this simplification is twofold: it conserves computational resources and reduces time expenditure.